

mindXtrain — GLM-5.1, aGLM Lineage & Training-Framework Master Reference

Foundational technical brief prepared for Gregory ("codephreak" / Professor Codephreak), BANKON / PYTHAI / DELTAVERSE, May 2026. Apache 2.0 redistribution target. Python $\geq$ 3.12. Podman, OpenBSD vmm, Foundry. Flat snake_case, cypherpunk2048 standard.

This document is the working reference for the mindXtrain project — the training framework that will produce aGLM v2 derivatives consumable by automindX v2 and the cognitive API at mindx.pythai.net. It corrects three premise errors in the original research brief, executes the full technical analysis the brief requested, and ends with a concrete construction plan and a non-trivial recommendation: **target Qwen3.5 as the primary base and treat GLM-5.1 as a premium specialist track**, on rigorous evidence laid out below. None of this is hand-waved; every figure is sourced and every disagreement between sources is flagged.

Three corrections to the brief, before anything else

The first correction is that **the GLM-5.1 "family" the brief assumed does not exist**. There is no GLM-5.1-Air, no GLM-5.1-Flash, no GLM-5.1-AirX, no GLM-5.1-Plus, and no GLM-5.1V. The Hugging Face collection at [zai-org](https://huggingface.co/zai-org) contains exactly two artifacts — [zai-org/GLM-5.1](https://huggingface.co/zai-org/GLM-5.1) (BF16 weights) and [zai-org/GLM-5.1-FP8](https://huggingface.co/zai-org/GLM-5.1-FP8) (native FP8 quantization). The Z.AI documentation sidebar lists a single GLM-5.1 entry. The Ollama library exposes one tag, [glm-5.1:cloud](https://ollama.com/library/glm-5.1:cloud), which routes to Z.AI's hosted endpoint rather than running locally. Earlier-generation tiers (GLM-4.5-Air, GLM-4.5-Flash, GLM-4.7-Flash, GLM-4.7-FlashX) remain accessible on the Z.AI API but do not share GLM-5.1's [glm_moe_dsa](https://ollama.com/library/glm-moe-dsa) architecture and are not part of the same training generation. The "Air/Flash" cost-tier idea from GLM-4.5 was deliberately abandoned for GLM-5.1: Z.AI shipped one flagship designed to do everything, plus an FP8 quantization to make it deployable on a single 8×B200 node. Any planning that depends on a smaller native GLM-5.1 variant has to either use community quantizations, fall back to GLM-4.5-Air for the cheap tier, or pick a different model family for the lower rungs of the size ladder. [\(VLLM + 2\)](#)

The second correction is that **the "GLM" inside aGLM is not Zhipu GLM**. Reading [pythaiml/automindx/aglm.py](https://pythaiml.com/automindx/aglm.py) and the [autoGLM/README-md](#) concept document together makes this unambiguous: aGLM stands for *Autonomous General Learning Model* (also rendered *Autonomous General Learning Machine* — both are author-sanctioned). The actual model loaded by [aglm.py](https://pythaiml.com/automindx/aglm.py) in its current canonical form is [TheBloke/llama2-7b-chat-codeCherryPop-qLoRA-GGML](https://huggingface.co/TheBloke/llama2-7b-chat-codeCherryPop-qLoRA-GGML), a Llama-2 GGML-era quant — not a ChatGLM or GLM-4 weight. The acronym collision with Zhipu's separate "AutoGLM" research project ([zai-org/Open-AutoGLM](https://huggingface.co/zai-org/Open-AutoGLM), the AutoGLM-Phone-9B paper) is a permanent disambiguation hazard the v2 README must address up front. mindXtrain v2's job is therefore *not* to "wrap GLM-5.1 inside aglm.py" in the literal sense; it is to upgrade the aGLM runtime so that one of the model backends it can dispatch to is GLM-5.1 (or Qwen3.5, or any modern base), while preserving the Codephreak persona,

agenda-conditioning, four-axis decomposition, and JSON-on-disk memory pattern that constitute aGLM's identity. (github)

The third correction is that (huggingface/ml-intern) is not a training framework. The repository, first published around 19–21 April 2026 by Aksel Joonas Reedi and the HF AI-Agents team, is a Claude-Code-style autonomous coding agent pre-wired to Hugging Face Hub, Papers, Datasets, Jobs, and Spaces. Its (pyproject.toml) declares (name = "hf-agent") at version 0.1.0 and lists (huggingface-hub>=1.0.1), (litellm>=1.83.0), (fastmcp>=3.2.0), (pydantic>=2.12.3), and (whoosh>=2.7.4) — but it does **not** depend on (transformers), (peft), (trl), (accelerate), (lighteval), or (evaluate). Distributed training is delegated entirely: ml-intern's (agent/tools/hf_jobs.py) writes a TRL-or-Transformers script in-context, submits it to a Hugging Face Job at the chosen flavor ((gpu-h100), (gpu-a100), etc.), polls the logs, and feeds them back into the LLM context. There is no Trainer abstraction inside ml-intern. There is also, as of late April 2026, **no LICENSE file** — issue #41 in the ml-intern repo is the open ticket asking HF to confirm; the third-party (mudler/universal-ml-intern) port assumes Apache 2.0, but that is an assumption, not an attached license. mindXtrain cannot vendor ml-intern code yet. What it **can** do — and what this document recommends — is study and reimplement five specific patterns from ml-intern (the unified (ToolRouter), the bounded ReAct loop with doom-loop detector, the 170k-token auto-compacting (ContextManager) with Claude-Code-JSONL trajectory upload, the approval-required tool flag with live USD pricing, and the three-phase Research → Plan → Implement system prompt). These patterns belong in the *operator layer* above mindXtrain's training core, not inside it. (MarkTechPost + 7)

With those corrections in place, the rest of this document is exhaustive on the technical substance.

Part 1 — GLM-5.1 in full technical depth

GLM-5.1 was announced via the Z.AI blog post "*GLM-5.1: Towards Long-Horizon Tasks*" ((https://z.ai/blog/glm-5.1)) and released on 7 April 2026. It is not a new pretraining run; it is a post-training refresh of GLM-5, sharing the same (glm_moe_dsa) architecture and the same paper, "*GLM-5: from Vibe Coding to Agentic Engineering*" (arXiv 2602.15763, lead author Aohan Zeng, 185-author roster). The differentiator that justifies the .1 bump, per the Z.AI blog and the model card on (huggingface.co/zai-org/GLM-5.1), is asynchronous agent-RL training "emphasizing hundreds of rounds and thousands of tool calls" — agentic-trajectory reinforcement learning with verifiable rewards on real engineering tasks (the Z.AI team cites optimizing a vector database to 21,500 QPS over 600+ iterations and 6,000 tool calls, and KernelBench Level 3 producing a 3.6× geometric-mean speedup vs (torch.compile max-autotune)'s 1.49× over thousands of optimization rounds). The marketing tagline — "can work autonomously on a single task for up to 8 hours" — is a deliberate framing of where the model's headroom is. (Build Fast with AI + 7)

Parameter accounting and shape

GLM-5.1 is a Mixture-of-Experts decoder of total capacity ~754 B parameters with ~40.8 B active per token. The 754 B figure is what the Hugging Face model card metadata reports; Lambda's deployment guide and the GLM-5 GitHub README report 744 B; the gap (~10 B) is reconciled by the static reference site (`glm51.si5.pl`) — built directly from the merged

`transformers/models/glm_moe_dsa/{configuration,modular,modeling}_glm_moe_dsa.py` — as MTP head plus fp32 buffers plus per-checkpoint scale tensors. Active-per-token is consistent across all sources at "~40 B". (LLM Stats + 4)

The model has 78 decoder layers. The first three are dense; the remaining seventy-five are MoE — the configuration object encodes this as `mlp_layer_types = ["dense"] * 3 + ["sparse"] * 75`. Hidden size (`d_model`) is 6,144. Vocabulary is 154,880 tokens (up from 151,552 in GLM-4.6); embeddings and `lm_head` are untied (`tie_word_embeddings=False`), so each is a $6,144 \times 154,880$ matrix of about 951.4 M parameters, contributing roughly 1.9 B parameters to the total just for the input/output projections. Native context length is 202,752 tokens — about 200 K — with no YaRN or NTK extrapolation in the released config. RoPE is the NeoX/Llama split-half variant (rotate-half), applied only to the 64-dimensional "rope" subspace via `attribute_map = {"head_dim": "qk_rope_head_dim"}; rope_theta` is configurable in `rope_parameters` and `partial_rotary_factor` is honored. Crucially, GLM-5.1 explicitly **removed** the interleaved-RoPE path GLM-5 inherited from DeepSeek V3 — `rope_interleave` raises `AttributeError`. Any weight-conversion script written for GLM-5 that assumes the DeepSeek V3 RoPE will silently break on GLM-5.1. (MarkTechPost)

Normalization is RMSNorm pre-norm, weight-only, $\epsilon = 1e-5$ throughout the body and inside the attention's `q_a_layernorm` and `kv_a_layernorm`. The lone exception is the DSA Indexer's `k_norm`, which is a standard `nn.LayerNorm` with $\epsilon = 1e-6$ — a deliberate departure to match the DeepSeek V3.2 reference implementation. Activations are SiLU inside a SwiGLU GLU MLP (`gate_proj`, `up_proj`, `down_proj`, no biases). Attention bias is False everywhere. The FP8 escapes are explicit: `_keep_in_fp32_modules = ["indexer.weights_proj"]` and `_keep_in_fp32_modules_strict = ["e_score_correction_bias"]`.

Attention: MLA stacked with DSA

Every layer combines two attention innovations. The first is Multi-head Latent Attention from DeepSeek V2/V3: 64 query heads, 64 KV heads (in MLA, KV are produced from a low-rank latent and expanded per head, so `num_key_value_groups` is 1 and `repeat_kv` is a no-op), a query LoRA rank of 2,048 (raised from 768 in GLM-5 because the q-latent must now also feed the new DSA Indexer), a KV LoRA rank of 512, an `nope` content slice of 192 dimensions per head and a `rope` positional slice of 64 dimensions per head for total `qk_head_dim = 256`, and a value head dimension of 256. The latent KV cache is therefore $512 + 64 = 576$ elements per token per layer, or roughly 89.9 KB per token over the 78 layers; the *expanded* KV cache that vanilla Transformers

materializes is $64 \times (256 + 256) = 32,768$ elements per token per layer, about 5.13 MB per token. That ratio is the entire reason MLA exists.

The second innovation is the DeepSeek Sparse Attention Indexer, the headline change from GLM-5. The Indexer has 32 heads of 128 dimensions each, takes the post-`(q_a_layernorm)` query latent for queries, reads raw `(hidden_states)` through its own `(wk)` projection for keys, computes scores as `(Σ_h weights[s,h] · ReLU(softmax_scale · q[s,h] · k[t]))` in fp32, and selects `(index_topk = 2,048)` keys per query. The Indexer adds about 9.4 M parameters per layer (~5.4% of the layer's attention weight) and maintains its own KV cache outside the standard `(DynamicCache)` as a layer-local `(cached_keys)` tensor (~256 B per token in bf16). The flash-mla kernel from `(kernels-community/flash-mla)` reads the `(topk_indices)` kwarg directly to skip masked positions; the eager and SDPA paths instead materialize a full `([B, S, T]) (-inf)` mask and `(scatter_)` zeros at the indexer-selected columns. The point of the Indexer is that it makes per-token attention compute **independent of sequence length** beyond about 2 K, which is what makes 200 K-token decode tractable at 754 B-parameter scale. Note that DSA does *not* shrink the KV cache — it shrinks compute. KV cache memory at long context is still dominated by MLA's latent representation.

MoE block

The 75 sparse layers carry 256 routed experts plus 1 always-on shared expert with intermediate size 2,048; top-k routing selects 8 routed experts per token, so $8 + 1 = 9$ experts are active at any moment. The dense FFN in layers 0-2 has intermediate size 12,288 (about $2 \times$ hidden). Router scoring uses sigmoid (not softmax) in fp32, with auxiliary-loss-free balancing via a per-expert `(e_score_correction_bias)` that is added for *selection only*, not for weighting — the DeepSeek V3 trick. `(routed_scaling_factor)` is 2.5 (raised from 1.8 in GLM-5), multiplied into the L1-normalized top-k sigmoid scores before they are added to the residual. The shared-expert path is added without the routing scale, so it acts as a constant baseline. The `(n_group)/(topk_group)` fields are 1/1, which collapses the multi-group routing back to plain top-8 over all 256 experts. Per-MoE-layer capacity is $256 \times 37.75 \text{ M} (3 \times 6144 \times 2048) + 1 \times 37.75 \text{ M} + 1.57 \text{ M router} \approx 9.70 \text{ B}$; per-MoE-layer active is $8 \times 37.75 \text{ M} + 37.75 \text{ M} \approx 341.3 \text{ M FFN} + 174.4 \text{ M attention} \approx 515.7 \text{ M per token per layer}$. Aggregate: ~743.6 B capacity, ~40.8 B active, matching the official "754 B / ~40 B" within the noted reconciliation gap.

MTP head, tokenizer, chat template

GLM-5.1 ships a Multi-Token Prediction head for speculative decoding, DeepSeek V3 style. The Hugging Face transformers integration uses `(_keys_to_ignore_on_load_unexpected = [r"model\.layers\.78.\*"])` to admit a layer-78 placeholder for the MTP head; the main body has 78 layers indexed 0-77. vLLM exposes MTP via `(--speculative-config.method mtp --speculative-config.num_speculative_tokens 3)`; SGLang uses EAGLE for the same role. The tokenizer is a SentencePiece-derived BPE with 154,880 vocabulary entries; special tokens include `(<|system|>)`, `(<|user|>)`, `(<|assistant|>)`, `(<|observation|>)`, `(<|tool|>)`, plus thinking

delimiters and tool-call tokens. vLLM's `--tool-call-parser glm47 --reasoning-parser glm45` flags select the right parser pair. **Thinking mode is enabled by default** in GLM-5.1 — a behavioral change from GLM-5 — and is disabled with `chat_template_kwargs: {"enable_thinking": false}`. The `chat_template.jinja` (~4.67 kB) supports Claude-style deferred tool loading via `defer_loading=True`, which puts tool schemas into the tool *result* messages rather than the system prompt, and accepts both `List[tool]` and `List[tool.function]` shapes for SGLang compatibility. There are two thinking flavors: *Interleaved Thinking* (default, general chat) and *Interleaved + Preserved Thinking* for agentic workflows like Claude Code, Roo Code, and Kilo Code, toggled via the `enable_thinking` and `clear_thinking` chat-template kwargs. (GitHub + 5)

Training methodology, with explicit gaps

Pretraining used 28.5 trillion tokens, up from 23 T for GLM-4.5. The English:Chinese:other split is not disclosed in the model card or the extractable arXiv abstract; the tokenizer is jointly built over English and Chinese with code and tool tokens, and the Hugging Face metadata tags the model with both `English` and `Chinese`. One secondary blog (`aimadetools.com`) claims pretraining was done on 100,000 Huawei Ascend 910B chips with zero NVIDIA dependency; this is **not** corroborated by the arXiv paper, the model card, or the Z.AI blog and should be treated as unverified reporting, although the parallel choice of `xLLM` (JD's Ascend-aware serving stack) as a first-class deployment target lends it some weight. Total pretraining FLOPs are not disclosed.

(GitHub) (Hugging Face)

Post-training is described in the arXiv 2602.15763 abstract — the only programmatically extractable text from the paper at the time of this research; the PDF body is not machine-readable in current archives — as using an asynchronous reinforcement-learning infrastructure called `slime`, open-sourced at `github.com/THUDM/slime`. `slime` decouples generation from training so that fine-grained RL iterations can run without blocking the trainer. The asynchronous agent-RL algorithms are said to "improve RL quality, enabling the model to learn from complex, long-horizon interactions more effectively." For GLM-5.1 specifically, the differentiator is that this RL was run with verifiable rewards over real engineering tasks at "hundreds of rounds and thousands of tool calls" depth. The choice of PPO vs GRPO vs DPO, the SFT data mix, the reward-model architecture, the cold-start reasoning data, and the source of any distillation are **not disclosed**. mindXtrain's RL track will therefore have to either trust `slime` as the operational primitive (it is GPL-3.0 / Apache 2.0 / MIT compatible — verify per repo file) or stand up its own GRPO/DPO loops via TRL. (arXiv + 4)

Benchmarks: vendor-claimed vs independent

Z.AI's published numbers for GLM-5.1 are deliberately concentrated on long-horizon agentic and engineering benchmarks — SWE-Bench Pro 58.4 (claimed SOTA, 0.7 points above GPT-5.4's 57.7 and 1.1 above Claude Opus 4.6's 57.3), Terminal-Bench 2.0 63.5 with Terminus-2 / 66.5 with Claude Code (vs Gemini 3.1 Pro at 68.5), CyberGym 68.7 (claimed SOTA over Claude Opus 4.6 at

66.6), BrowseComp 68.0 (claimed SOTA), MCP-Atlas 71.8, τ^3 -Bench 70.6, Tool-Decathlon 40.7, Vending Bench 2 final balance \$5,634, AIME 2026 95.3, HMMT February 2026 82.6, GPQA-Diamond 86.2, HLE 31.0 / HLE w/ Tools 52.3. Z.AI explicitly did not publish numbers for MMLU, MMLU-Pro, CMMLU, C-Eval, BBH, MATH-500, GSM8K, HumanEval, MBPP, LiveCodeBench, BigCodeBench, BFCL v3, AgentBench, GAIA, vanilla SWE-bench, IFEval, Arena-Hard, RULER, or NIAH. If mindXtrain needs any of those, they have to be re-run locally. (ModemGuides +2 + 5)

The independent picture is more mixed. Artificial Analysis's Intelligence Index v4.0 places GLM-5.1 (Reasoning) at 51 and GLM-5.1 (Non-Reasoning) at 44 — well above the open-weight reasoning median of 29 but a clear step below GPT-5.4 / Gemini 3.1 Pro / Claude Opus 4.6, which sit closer to 65–75. AAI v4.0 evaluation consumed about 110 M output tokens (vs a 40 M peer median, "very verbose") at \$543.95 total. BenchLM's *provisional* leaderboard ranks GLM-5.1 #14 of 115 models with an overall 83 — but its *verified* leaderboard, which only counts source-attached scores, places GLM-5.1 #21 of 23. That gap is the cleanest signal that vendor-reported scores are running ahead of independently re-run scores. LiveBench had not yet ingested GLM-5.1 in the captures available at the time of research; LMSYS Chatbot Arena had not surfaced a GLM-5.1 ELO yet (GLM-5 base sits around 1451 and was reported as the #1 open model in the GLM-5 paper); HuggingFace Open LLM Leaderboard v2 cannot evaluate models of this size under its compute budget. The Scale AI SEAL leaderboard hosts the SWE-Bench Pro 58.4 number with the asterisk denoting self-submission and no independent re-run as of this report. The fair characterization is that GLM-5.1 is genuinely a top-tier open-weights model that sits within roughly 5–10% of the absolute frontier closed models on most coding benchmarks, narrowly leads on SWE-Bench Pro (self-reported), and is the strongest open agentic-coding model available in May 2026 — but the Z.AI marketing claim of "outperforming GPT-5.4, Claude Opus 4.6, and Gemini 3.1 Pro" is true on a single benchmark and false on most others. Cherry-picked headline. (Artificial Analysis + 6)

Inference characteristics and VRAM math worked out

Weight memory at 754 B parameters: BF16 $\approx$ 1,508 GB (what `zai-org/GLM-5.1` ships, marked "BF16 · F32" in the HF metadata); native FP8 $\approx$ 754 GB (what `zai-org/GLM-5.1-FP8` ships — the on-disk checkpoint is 756 GB across 142 safetensors shards of about 5.36 GB each); INT4 $\approx$ 377 GB (community quant, currently one such model listed under HF "Quantizations"); INT8 $\approx$ 754 GB (rare — not commonly distributed). GGUF Q4_K_M / Q5_K_M / Q6_K / Q8_0 are **not available** because llama.cpp does not yet ship the `glm_moe_dsa` architecture; Unsloth has shipped Dynamic 2.0 GGUF quant in the UD-IQ2_M (~241 GB) through UD-Q8_0 range at `huggingface.co/unsloth/GLM-5.1-GGUF` by carrying a ggml-org/llama.cpp build, but the upstream PR numbers landing the architecture were not surfaced in the research pass and the architecture is closely related to GLM-4.5/4.6. (MarkTechPost) (Unsloth AI)

KV cache at batch 1 in BF16, expanded form: about 3.7 GB at 4 K tokens, 30 GB at 32 K, 120 GB at 128 K, 187 GB at 200 K. In MLA-compressed (latent) form: 0.36 GB at 4 K, 2.9 GB at 32 K, 11.5 GB at 128 K, 18 GB at 200 K. Total VRAM for FP8 weights plus KV at 128 K context: 754 + 120 = 874

GB (expanded) or $754 + 11.5 = 765$ GB (MLA-compressed). For INT4 + MLA-compressed KV: $377 + 5.7 = 383$ GB. With DSA active on a ~2 K window, FP8 + 0.18 GB = 754 GB. The official Lambda minimum is a **single 8×B200 (HGX B200) node** to load the FP8 model with usable context. **An RTX 4090 24 GB cannot run GLM-5.1 at any quantization. An M3 Ultra 192 GB unified-memory Mac cannot run the full model at any quantization** — the FP8 weights alone are 754 GB; in principle the INT4 quant would fit if MLX support existed, but `ml-explore/mlx-lm` issue #879 ("Add model support for GLM-5 (`glm_moe_dsa`) architecture") was still open at research time. **An M2 Max 96 GB cannot run any usable form of GLM-5.1.** A100 80 GB requires roughly 10× to load FP8. `Lambda` `ModemGuides`

Throughput, from Lambda's official benchmarks at 8 K input / 2 K output and 32 concurrent users: SGLang 0.5.10 on 1× HGX B200 produces 1,345.4 output tokens/s, 42.0 per-user, 6,727.2 total, with mean TTFT 1,073 ms and mean inter-token latency 58.6 ms. vLLM 0.19.0 on the same hardware produces 1,265.4 / 39.5 / 6,327.2 tokens/s with TTFT 1,317 ms, ITL 57.8 ms. Artificial Analysis's median across providers is 53.7 tok/s output with TTFT 1.73 s — slower than the open-weight median of 56.3 tok/s in this scale class. `Artificial Analysis` `Artificial Analysis`

The recommended decoding parameters from the Z.AI model card are: temperature 1.0, top_p 0.95, max generation 131,072 for default and general use; temperature 0.7, top_p 1.0, max 16,384, with `enable_thinking=true` and `clear_thinking=false` for Terminal-Bench / coding-agent flows; temperature 0, max 16,384, Interleaved + Preserved thinking for τ^2 -Bench-style pure tool-use flows. `Hugging Face`

Framework support matrix

Hugging Face Transformers requires version 5.3.0 or later — the `glm_moe_dsa` architecture is not in transformers 4.x. (Z.AI's README states "v0.5.3+", which is a typographical error for v5.3.0+; Lambda's deployment guide uses the corrected version.) `trust_remote_code` is no longer required after the merge. vLLM supports GLM-5.1 from 0.19.0 with custom Docker images at `vllm/vllm-openai:glm51` and `vllm/vllm-openai:glm51-cu130` (CUDA 13+); the recipes URL is `https://docs.vllm.ai/projects/recipes/en/latest/GLM/GLM5.html`. There is a known caveat: tool-calling combined with MTP-enabled speculative decoding requires the vLLM main branch rather than the 0.19.0 release. SGLang requires v0.5.10 (not 0.5.10rc0, which has a known flashmla bug fixed in the 0.5.10 release). xLLM v0.8.0+ supports the Ascend NPU path. KTransformers v0.5.3+ supports CPU-offload-plus-GPU hybrid. Ollama supports `glm-5.1:cloud` (cloud-only) as of research time; an open issue at `github.com/ollama/ollama/issues/15412` tracks offline support. A community fork at `ollama.com/frob/glm-5.1` imports `unsloth/GLM-5.1-GGUF` and requires a patched Ollama build with PR #14864 applied; the maintainer notes tool-calling is poor on this fork pending an Ollama PARSER. **Important known bug:** Unsloth's discussion thread on `huggingface.co/unsloth/GLM-5.1-GGUF/discussions/4` warns that CUDA 13.2 produces gibberish or breaks tool calling on Gemma 4 and GLM-5.1; NVIDIA had not issued a fix at research time. Use CUDA 13.0 or 13.1 for any GGUF deployment. `Lambda + 6`

A canonical vLLM launch and a canonical SGLang launch, copy-pasteable:

bash

```
vllm serve zai-org/GLM-5.1-FP8 \
  --tensor-parallel-size 8 \
  --max-model-len 202752 \
  --max-num-seqs 64 \
  --speculative-config.method mtp \
  --speculative-config.num_speculative_tokens 3 \
  --tool-call-parser glm47 \
  --reasoning-parser glm45 \
  --enable-auto-tool-choice \
  --chat-template-content-format=string \
  --served-model-name glm-5.1-fp8
```

bash

```
SGLANG_ENABLE_SPEC_V2=1 \
sglang serve \
  --model-path zai-org/GLM-5.1-FP8 \
  --tp-size 8 \
  --tool-call-parser glm47 \
  --reasoning-parser glm45 \
  --speculative-algorithm EAGLE \
  --speculative-num-steps 3 \
  --speculative-eagle-topk 1 \
  --speculative-num-draft-tokens 4 \
  --mem-fraction-static 0.85 \
  --served-model-name glm-5.1-fp8
```

API access

The English portal is (<https://api.z.ai/api/paas/v4/>), with OpenAI-compatible chat completions at (`POST /chat/completions`), (`Authorization: Bearer <key>`), keys managed at (<https://z.ai/manage-apikey/apikey-list>). The Chinese portal is (bigmodel.cn). The model ID is (`glm-5.1`); siblings include (`glm-5`), (`glm-5-turbo`), (`glm-4.7`), (`glm-4.7-flash`), (`glm-4.7-flashx`), (`glm-4.6`), (`glm-4.5`), (`glm-4.5-air`), (`glm-4.5-airx`), (`glm-4.5-x`), and (`glm-4.5-flash`). The Python SDK is (`pip install zai-sdk`) ($\geq 0.2.2$); Java is Maven (`ai.z.openapi:zai-sdk:0.3.3`); any OpenAI-compatible client with (`base_url="https://api.z.ai/api/paas/v4/"`) works. Pricing for GLM-5.1 in USD per 1 M tokens is \$1.40 in / \$0.26 cached input / \$4.40 out, with cache storage free for a limited time. Third-party reseller pricing varies: OpenRouter ([z-ai/glm-](https://openrouter.ai/providers/zai-glm-5.1)

`5.1` is \$1.05 / \$3.50 (output capped at 65,535 tokens); Requesty / Fireworks match the official \$1.40 / \$4.40 (max output 25,000, 202 K context with prompt caching "up to 90%"); Inworld / DeepInfra match OpenRouter at \$1.05 / \$3.50; the Artificial Analysis median is \$1.40 / \$4.40. Feature flags supported on the native API include OpenAI-compatible `tools` arrays (with Claude-style deferred tool loading), MCP integration, structured JSON output, streaming including tool-streaming output, prefix/context caching, the `thinking: {"type": "enabled" | "disabled"}` toggle, and a built-in web search tool at \$0.01 per use. Vision input is **not** supported on `glm-5.1` — use the sibling `glm-5v-turbo`. There is no free tier on `glm-5.1` itself; the free tier exists on the Flash variants of earlier generations. (Constellation Research + 5)

License — the most important section for BANKON

GLM-5.1 weights ship under the **MIT License** on Hugging Face. The model card YAML for both `zai-org/GLM-5.1` and `zai-org/GLM-5.1-FP8` declares `license: mit`; the Unsloth GGUF mirror at `huggingface.co/unsloth/GLM-5.1-GGUF` does the same; Wikipedia summarizes Z.ai's policy as "released under the free and open-source MIT License since July 2025." The Hugging Face license-label "mit" maps in HF's taxonomy to the canonical SPDX MIT text — HF rejects uploads that declare `license: mit` if the LICENSE file deviates. Treat this as high-confidence even though the literal raw bytes of `github.com/zai-org/GLM-5/blob/main/LICENSE` returned 429s during the research pass. (Note: there is a code-vs-weights split — the GitHub repo header for `zai-org/GLM-5` reads "Apache License 2.0" in the search index sidebar, which means the *code* in the repo is Apache 2.0 while the *weights* on Hugging Face are MIT. This is the same pattern Z.ai used for GLM-4.5 and GLM-4.6.) (ModemGuides + 3)

Verbatim, the operative MIT text for the weights is:

MIT License

Copyright (c) 2026 Z.ai (or "ZHIPU AI" as published)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Clause-by-clause: **commercial use is unrestricted**; the grant explicitly covers "use, copy, modify, merge, publish, distribute, sublicense, and/or sell." **Redistribution is unrestricted** subject to preserving the copyright notice and the permission text. **Derivative works can be released under any license BANKON chooses**, including Apache 2.0 — MIT is permissive, non-copyleft, and does not propagate. **Attribution is the MIT notice preservation, nothing more** — no name-prefix requirement, no "Powered by GLM" badge, no UI credit. **There are no field-of-use restrictions on the weights** — no military prohibition, no surveillance carve-out, no critical-infrastructure exclusion, no biosecurity rider. There is no patent grant (this is MIT's standard weakness vs Apache 2.0; patent rights are at most implied), no trademark grant (so do not use "GLM", "Zhipu", or "Z.ai" marks in BANKON product names in ways that suggest endorsement — naming the derivative `aGLM-BANKON` is borderline, and a more defensible product mark in user-facing UI is something like "MindX" with a model-card technical name of `aGLM-BANKON`), no AUP, no MAU thresholds, and no geographic restrictions encoded in the license itself. Z.ai is on the U.S. Commerce Department Entity List as of January 2025 — that is a sanctions/export-controls matter affecting U.S. companies *transacting with* Z.ai as a corporate counterparty, not a license restriction encoded in the MIT text; the consensus practice is that downloading MIT-licensed open weights is not Entity-List-restricted activity, but BANKON should confirm with counsel. `Build Fast with AI + 2`

The hosted-API regime is separate. (<https://docs.z.ai/legal-agreement/terms-of-use>) governs ([api.z.ai](#)), ([BigModel.cn](#)), and ([chat.z.ai](#)) and contains AUP-style language including a prohibition on using outputs "for the development, training, labeling, fine-tuning, optimization, iteration, or similar activities related to external models" or to "develop, train, or enhance algorithms or models that compete with us"; restrictions on services "requiring subject qualification, including but not limited to medical services, legal services" plus "any decision-making behavior, operation of critical infrastructure, transportation technologies, heavy machinery"; and a clause stating "If you suffer damage after training, fine-tuning and development and claim that we should bear the responsibility, you shall prove that the damage is unrelated to your training, fine-tuning and development, otherwise we shall be exempted from liability for the damage." These restrictions are **API-only** and do not bind BANKON if it self-hosts the open weights. ([Overview + 4](#))

The compliance recipe for BANKON is therefore: pull the FP8 weights from (huggingface.co/zai-org/GLM-5.1-FP8), self-host on owned/leased GPUs (vLLM or SGLang on a single 8×B200 or two 8×H100s), and avoid touching ([api.z.ai](#)) for any production path that would create a contractual nexus. For the ([aGLM-BANKON](#)) derivative published under Apache 2.0, ship a ([LICENSE](#)) file containing the Apache 2.0 text, ship a ([NOTICE](#)) file containing the verbatim upstream MIT notice ("Copyright (c) 2026 Z.ai. Licensed under the MIT License. See LICENSE-MIT-upstream for full text."), ship ([LICENSE-MIT-upstream](#)) with the MIT text and Z.ai's copyright assertion, and stamp the model card with "Copyright (c) 2026 BANKON — all rights reserved. Fine-tuned from (zai-org/GLM-5.1), originally licensed under the MIT License (Copyright (c) 2026 Z.ai)." Dual-licensing is permitted; there is no copyleft pull-through. The derivative does not have to inherit the MIT licence — only the upstream notice must be preserved.

Compared to peers: Qwen3 ships under Apache 2.0 across the entire family — equivalent for redistribution, slightly stronger on patent posture, slightly heavier on NOTICE compliance overhead. DeepSeek V3.2 / R1 are MIT for both code and weights — equivalent. Phi-4 / Phi-4-mini are MIT — equivalent. Llama 3.3 / Llama 4 ship under the Llama Community License with the 700 M MAU clause, the "Built with Llama" name-prefix and badge requirement, and the AUP at (llama.meta.com/llama3_3/use-policy) incorporated by reference — *incompatible* with clean Apache 2.0 downstream redistribution. Gemma 1–3 carried the Gemma Terms of Use with a Prohibited Use Policy that propagates to downstream redistributors — restrictive — but **Gemma 4** (April 2026) flipped to Apache 2.0, becoming compatible. Mistral's flagship line (Mistral Large, Codestral, Mistral Small) used the Mistral Research License (non-commercial) until **Mistral Large 3** (December 2025), which moved to Apache 2.0 along with Minstral 3 and Mistral Small 4. Cohere Command A is CC-BY-NC 4.0 with an Acceptable Use Addendum — non-commercial only — *incompatible*. Falcon 3 ships under TII Falcon License 2.0, Apache-2.0-derived but with an enforceable AUP — most legal teams will treat this as restrictive. ([Qwen + 2](#))

Part 2 — autoGLM and aGLM lineage as foundational context

The pythaiml/automindx repository at (<https://github.com/pythaiml/automindx>) (id 686099738, network root for the GATERAGE fork) is 84.0% Python, 13.3% Shell, 2.7% Dockerfile, with 23 commits on `main`, 2 stars, 4 forks, and a flat layout: `aglm.py`, `automind.py`, `memory.py`, `uiux.py`, paired with `algm.md`, `automind.md`, `memory.md`, `uiux.md`, plus `4096chunk.md`, `INSTALL.md`, `DOCUMENTATION.md`, `Dockerfile`, `LICENSE`, `README.md`, `automindx.install`, `chunk4096.py`, `hfUIUX.py`, `hfapp.py`, `hfmemory.py`, and `requirements.txt`. The README declares the project's core composition as `codephreak = uiux.py + memory.py + automind.py + aglm.py` and states the persona explicitly: *"Professor Codephreak is an expert in machine learning, computer science and computer programming."* The runtime entry point is `python3 uiux.py --model_name="TheBloke/llama2-7b-chat-codeCherryPop-qLoRA-GGML" --tokenizer_name="TheBloke/llama2-7b-chat-codeCherryPop-qLoRA-GGML" --model_type="ggml" --save_history --file_name="llama-2-7b-chat-codeCherryPop.ggmlv3.q4_1.bin"`. The license is GPL-3.0 by inheritance (the GATERAGE/aglm fork is explicitly GPL-3.0; the wider codephreak ecosystem — easyAGI, RAGE, MASTERMIND — is uniformly GPL-3.0; the README footer asserts "MASTERMIND (c) codephreak GPLv3 2024"); the LICENSE bytes were not retrievable in the research pass but the inheritance is unambiguous. ([github + 4](#))

`aglm.py` itself is a Hugging Face Transformers wrapper, not an agent. The imports — reconstructed from the Hugging Face `aGLM` org card, which is an authoritative author paraphrase — are `os`, `glob`, `ujson`, `psutil`, `transformers.AutoModelForCausalLM`, `transformers.AutoTokenizer`, and `automind.format_to_llama_chat_style`. The single class is `LlamaModel(model_name, models_folder)`, with methods `initialize_model()` (loads tokenizer and model from `models_folder + model_name` via `AutoTokenizer.from_pretrained` and `AutoModelForCausalLM.from_pretrained`) and `generate_contextual_output(conversation_context)` (which formats via `format_to_llama_chat_style`, tokenizes, runs `model.generate(...)`, and decodes). Module-level helpers are `determine_batch_size()` (uses `psutil.virtual_memory()` against a hard-coded `MAX_MEMORY_USAGE` to decide how many memory-file JSONs to load per batch) and `main()` (globs `memory/*.json`, batches via `determine_batch_size()`, reads with `ujson`, builds `conversation_context`, calls `LlamaModel.generate_contextual_output(context)`, prints). There is no async, no asyncio, no LangChain, no SuperAGI, no AutoGen, no `openai`, no `anthropic`, no `chromadb`, no `faiss`, no `pgvector`. Memory is `glob`-walked from `*.json` files via `ujson`; persistence happens via `memory.save_conversation_memory(...)` writing timestamped JSON files. ([huggingface + 3](#))

The patterns that mindXtrain v2 must preserve are precise. First, the **four-axis decomposition**: `uiux.py` (interface) plus `memory.py` (persistence) plus `automind.py` (prompt/format) plus `aglm.py` (model). Second, the **`.py` paired with `.md` discipline** — every Python module has a

sibling Markdown documentation file colocated. Third, **shallow flat class hierarchy**:

(`LlamaModel`), (`DialogEntry`), (`EasyAGI`), (`AGI`), (`LogicTables`), (`SocraticReasoning`), (`SelfHealingSystem`), (`BDI`), (`Memory`) — no mixins, no ABCs, no Protocols; one responsibility each, instantiated once. Fourth, **synchronous default surface** —

(`LlamaModel.generate_contextual_output`) is sync, (`EasyAGI.main_loop`) is sync, the whole stack is a blocking REPL or batch. Fifth, **named-class registry dispatched by string key** for tool-like backends ((`GPT4o`), (`GroqModel`), (`OllamaModel`) selected by (`APIManager`) in easyAGI). Sixth, **append-only JSON-on-disk memory with batch-glob replay** ordered by filename timestamp. Seventh, **psutil-driven memory budgeting** generalizable to a (`ResourceBudget`) helper used identically by training, eval, and inference. Eighth, **CLI flags with no config file** — argparse in (`uiux.py`), with (`automindx.install`) shell-script baking the canonical invocation. Ninth, **persona-as-agenda**: the system prompt isn't a role description but contains an explicit *agenda* (the model is told "your job is to build the automindx deployment environment"). The agenda string must remain a first-class field in v2, not baked into a string. ([github](#))

The gaps mindXtrain must fill are equally precise. The hard-coded Llama-2 chat assumption via (`format_to_llama_chat_style`) must be replaced by a (`ChatTemplate`) abstraction that picks the right template for Llama-3, Mistral, Qwen, GLM-4, GLM-5.1, and any future base, falling back to (`tokenizer.apply_chat_template`). There is no multi-model registry — (`LlamaModel`) is one class for one family, with a separate dual-path GGML loader implicit in (`uiux.py`)'s `--model_type="ggml"` branch; v2 needs (`class ModelRegistry`) with (`register(name, factory)`) and (`get(name)`), supporting backends (`hf-transformers`), (`llama-cpp-python`), (`ollama`), (`vllm`), (`openai`), (`anthropic`), (`groq`), and the Z.ai API. The (`autoGLM/litellm`) fork already foreshadows this; wire it. There is no fine-tuning support — (`aglm.py`) does inference only — and the (`autoGLM/levanter`) fork (Apache 2.0, JAX + named tensors) was added as the latent training rail but never integrated. There is no evaluation harness, no observability, no memory layer beyond JSON-on-disk, no tokenizer-aware truncation (the (`chunk4096.py`) 4096-char ceiling is a workaround for context-window saturation), no session/concurrency primitives, no agent loop in (`aglm.py`), no prompt-as-data declarative persona files, no CI, and no tests. Each gap is a real technical debt item, not a feature wishlist.

The wider org context matters because it dictates v2's integration surface. Under (`autoGLM/`), the active source repos are (`autoGLM/easyAGI`) (Python, GPL-3.0, the openmindx → easyAGI point-of-departure stack with modules (`EasyAGI`), (`AGI`), (`LogicTables`), (`Reasoning`), (`SocraticReasoning`), (`SelfHealingSystem`), (`Memory`), (`GPT4o`), (`GroqModel`), (`OllamaModel`), (`APIManager`), (`BDI`)); (`autoGLM/funAGI`) (the first "working" instance with (`EasyAGI.main_loop`), archived as the canonical reference); (`autoGLM/automindx`) (org-level mirror of the canonical (`pythaiml/automindx`)); and (`autoGLM/README-md`), the canonical aGLM concept document defining the architecture as supervised+unsupervised learning with subsystems RAGE (retrieval-augmented memory), machine dreaming, MASTERMIND (logic+prediction), blockchain-anchored knowledge "THOTs" (Theories of Hypothetical Output Trajectories) on decentralized storage, and Continuous Adaptation and Optimization (auto-tuning / self-healing).

Forked-in tooling includes `autoGLM/RAGE` (GPL-3.0, retrieval engine), `autoGLM/imaginarium` (TypeScript, NLP UI), `autoGLM/pgvectorscale` (Rust, PostgreSQL-license, the intended long-term-memory backend), `autoGLM/litellm` (the OpenAI-format multi-LLM router), `autoGLM/levanter` (Apache-2.0, JAX-based scalable training — *the latent fine-tuning rail that was never wired into aglm.py*), and `autoGLM/anything-llm` (the desktop/Docker RAG+agent UI candidate). The most complete public surface of the aGLM concept lives at `GATERAGE/aglm`, which is a fork of `pythaiml/automindx` plus MASTERMIND modules (`prediction.py`, `nonmonotonic.py`, `socratic.py`, `reasoning.py`, `logic.py`, `epistemic.py`, `autonomize.py`, `bdi.py`, `terminai.py`, `terminai_module.py`, `SimpleCoder.py`, `model_handler.py`, `controller.py`, `config.json`, `config.py`, `main.py`), plus seventeen numbered `UIUX*.py` evolution snapshots) and is flagged in its own README as "currently BROKEN and useful as reference point of aGLM MASTERMIND and RAGE for modular component display only."

The integration topology is therefore: mindX (`github.com/abaracadabra/mindX`), augmentic-intelligence orchestration, public face (`mindx.pythai.net`) is the consumer of the aGLM v2 inference API; PYTHAI hosts the canonical `automindx`, `funAGI`, `pgvectorscale` and the domain anchors `ai.pythai.net`, `gpt.pythai.net`, `rage.pythai.net`, `bankon.pythai.net`, `agenticplace.pythai.net`; MASTERMIND is the rational-controller layer above aGLM (intended composition `MASTERMIND(aGLM, RAGE, BDI)`); RAGE / GATERAGE provides retrieval-augmented memory; DELTAVERSE supplies decentralized/metaDAO settlement; BANKON supplies banking-OS infrastructure (`bankonOS`, `BANKONPYTHAI`) with the Algorand ASA `203977300`, the ERC-8004 Identity Registry at `0x8004A169FB4a3325136EB29fA0ceB6D2e539a432` and Reputation Registry at `0x8004BAa17C55a88189AE136b182e5fdA19dE9b63`); Lighthouse Storage / Filecoin is the decentralized-knowledge-storage target named in `autoGLM/README-md`; and `cypherpunk2048/x402` provides the HTTP-402 micropayment protocol for paid inference settlement. mindXtrain produces `aGLM-BANKON` checkpoints, anchors their provenance via ERC-8004 attestations and Lighthouse-Filecoin CIDs, and ships them into automindx v2 which is consumed by mindX at `mindx.pythai.net`.

Part 3 — `huggingface/ml-intern` patterns to adopt for the operator layer

The five highest-leverage patterns from ml-intern, all of them in the *operator* layer above the trainer, are these.

The `ToolRouter` and `ToolSpec` dispatcher (`agent/core/tools.py`, ~230 LOC) unifies built-in async handlers, MCP JSON-RPC, and OpenAPI specs behind one OpenAI-compatible tool schema, with a deny-list (`{ "hf_jobs", "hf_doc_search", "hf_doc_fetch", "hf_whoami" }`) that prevents MCP-supplied tools from shadowing optimized built-ins, and graceful MCP-error capture that returns errors as strings rather than bubbling exceptions. The dataclass is

```
@dataclass class ToolSpec: name: str; description: str; parameters: dict; handler: Callable | None; needs_approval: bool = False
```

mindXtrain's heterogeneous backends —

start-run, eval-checkpoint, push-to-hub, deploy-to-API, anchor-to-IPFS, mint-ERC-8004-attestation — map cleanly onto this single surface. [DeepWiki](#) [DeepWiki](#)

The **bounded ReAct loop with explicit doom-loop detector** ([agent/agent_loop.py](#)), [submission_loop](#) and [Handlers.run_agent](#)) caps autonomous iterations at 300 and runs [DoomLoopDetector.observe\(resp.tool_calls\)](#) after each LLM turn; when repeated tool-call patterns (same tool, same args N times) are detected, a corrective system message is injected into the next call to break the cycle. Without this, autonomous training agents wedge inside the first hour of a long run. [GitHub](#)

The [ContextManager](#) with 170 K auto-compaction and Claude-Code-JSONL trajectory upload ([agent/core/context.py](#)) handles two responsibilities: it triggers [MessageCompactor](#) summarization when the message buffer crosses ~170 K tokens, and it serializes every session in Claude-Code JSONL format and pushes it to a private Hugging Face Dataset ([{username}/ml-intern-sessions](#)). HF's Agent Trace Viewer auto-renders these. For mindXtrain, the JSONL format is the right schema for an auditable, replayable, RFT-corpus-ready training-record format; just swap the storage backend from HF Dataset to a [StorageProvider](#) interface that writes equally to local FS, HF Datasets, IPFS, or Lighthouse-Filecoin.

The **approval-required tool flag with live USD pricing** combines a per-[ToolSpec](#) [needs_approval: bool](#) flag with a generic [await session.await_approval\(tc\)](#) flow wired to interactive CLI prompt, web "approve in browser" button, and Slack interactive approval. Live USD/hour pricing is surfaced at the prompt before the user approves any paid operation. For mindXtrain, this is exactly the right ergonomics for "agent picks GPU flavor → user sees \$/hour → user confirms → job runs → logs streamed back into context."

The **three-phase Research → Plan & Validate → Implement system prompt** ([agent/prompts/system_prompt_v3.yaml](#)) is the load-bearing discipline document. It forces a research phase (read papers, fetch documentation, enumerate options), a plan-and-validate phase (does this dataset exist, does this tokenizer match, does this flavor have the VRAM), and only then an implement phase. The single biggest reason ml-intern's GPQA demo completes in under ten hours is that the LLM is *not allowed* to call paid tools without first emitting and getting approval on a plan. mindXtrain's training agent must inherit this prompt verbatim, with the validation checklist extended to include base-model-hash-pinning, dataset-CID-pinning, and tokenizer-vocab-hash matching against the dataset's tokenized cache.

What ml-intern does **not** provide and mindXtrain must add: decentralized storage (no Filecoin/Lighthouse abstraction — everything goes to HF Hub); blockchain provenance (no on-chain attestation, no signed run-manifests, no Merkle-rooted training-data commitments); first-class hyperparameter sweeps (no Optuna, no Ray Tune — ablations are LLM-driven re-launches); BFCL and agentic-trajectory evals (the [eval](#) extra ships [inspect-ai>=0.3.149](#)) but BFCL is not first-class); auto-populated model cards from training runs (the agent can be prompted to write a card but there is no card-templating that reads [TrainingArguments](#) + eval

JSON); a hot-swap deployment plane to a live API (push-to-hub exists, but atomic API-level model swap with canary and rollback does not); a checkpoint registry with diffing and promotion; a typed `TrainingRun` Pydantic record as the canonical unit of provenance. All of these go into the mindXtrain trainer and observability layers, not the agent layer. [GitHub](#)

Part 4 — mindXtrain technical specification

mindXtrain is the production-grade training framework that produces `aGLM-BANKON-*` derivatives. It is built on the cypherpunk2048 standard (no proprietary lock-in, Podman over Docker, OpenBSD vmm over VirtualBox, Foundry as canonical Solidity test framework, flat snake_case layout, Python ≥ 3.12 , Apache 2.0 license with `((c) 2026 BANKON - all rights reserved)` plus upstream MIT NOTICE preservation). The framework has three layers: the **trainer core** (Trainer + TRL + PEFT + Accelerate + Transformers, hand-built), the **operator layer** (ml-intern-pattern-derived agent runtime, reimplemented), and the **provenance layer** (Lighthouse-Filecoin storage, ERC-8004 attestation, Pydantic `TrainingRun` records).

Repository layout

```
mindxtrain/
├─ pyproject.toml           # name="mindxtrain", py>=3.12, Apache-2.0
├─ LICENSE                 # Apache 2.0
├─ NOTICE                 # BANKON copyright + upstream MIT (Z.ai for
GLM-5.1, Apache for Qwen3.5)
├─ LICENSE-MIT-upstream-glm51  # verbatim Z.ai MIT notice
├─ README.md
├─ CHANGELOG.md
├─ Containerfile           # Podman, not Dockerfile
├─ compose.yaml           # podman-compose
├─ mindxtrain/
│   └─ __init__.py
│   └─ config/             # JSON config with ${ENV} interpolation, ml-
intern style
│   │   └─ train_default.json
│   │   └─ eval_default.json
│   │   └─ deploy_default.json
│   │   └─ schema.py       # Pydantic schemas
│   └─ data/              # data pipeline
│       └─ curate.py       # source → raw
│       └─ dedupe.py       # MinHash + exact-match
│       └─ filter.py       # quality, language, toxicity
│       └─ tokenize.py     # tokenizer-aware
│       └─ pack.py         # sequence packing
│       └─ synth.py        # synthetic data via GLM-5.1 / Qwen3.5
```


		└─ verify.py	# hash + manifest
		└─ models/	# model registry
		└─ registry.py	# ModelRegistry, register/get
		└─ chat_template.py	# ChatTemplate abstraction
		└─ glm51.py	# backend
		└─ qwen35.py	# backend
		└─ deepseek_v32.py	# backend
		└─ mistral3.py	# backend
		└─ phi4_mini.py	# backend
		└─ train/	# trainer core
		└─ sft.py	# full SFT + LoRA + QLoRA
		└─ dpo.py	# DPO via TRL
		└─ grpo.py	# GRPO via TRL
		└─ rlhf.py	# PPO via TRL
		└─ tool_use.py	# BFCL-style tool-trajectory training
		└─ distributed.py	# accelerate / FSDP / DeepSpeed config builders
		└─ callbacks.py	# eval-during-training, checkpoint mgmt
		└─ eval/	# eval harness
		└─ lighteval_adapter.py	
		└─ inspect_ai_adapter.py	
		└─ bfcl.py	# BFCL v3 / v4
		└─ persona_regression.py	# Codephreak voice tests
		└─ agenda_regression.py	# agenda-conditioning tests
		└─ tau_bench.py	
		└─ card.py	# auto model card from run
		└─ operator/	# ml-intern-pattern-derived
		└─ tool_router.py	# ToolRouter + ToolSpec
		└─ agent_loop.py	# bounded ReAct, doom-loop
		└─ context.py	# 170k compaction
		└─ trajectory.py	# JSONL writer
		└─ approval.py	# CLI/web/Slack approval flow
		└─ prompts/	
		└─ system_v1.yaml	# Research → Plan → Implement
		└─ codephreak.yaml	# persona + agenda, prompt-as-data
		└─ storage/	# provenance layer
		└─ provider.py	# StorageProvider interface
		└─ local_fs.py	# always-available fallback
		└─ hf_hub.py	
		└─ lighthouse.py	# Lighthouse Storage / Filecoin
		└─ ipfs.py	# raw IPFS
		└─ provenance/	# blockchain anchoring
		└─ manifest.py	# Pydantic TrainingRun record
		└─ erc8004.py	# Identity + Reputation registry attestation
		└─ algorand.py	# BANKON ASA 203977300 hooks

```

| | └─ x402.py # HTTP-402 micropayment for paid inference
| └─ deploy/ # automindX v2 integration
| | └─ registry.py # model-version registry
| | └─ hot_swap.py # atomic swap with canary
| | └─ ab_test.py # A/B traffic split
| | └─ api_client.py # mindx.pythai.net OpenAI-compatible client
| └─ budget/ # ResourceBudget (psutil-derived from aGLM)
| | └─ resource.py
| └─ cli/ # snake_case entries
| | └─ main.py # mindxtrain.cli.main:cli
└─ contracts/ # Foundry, for ERC-8004 hooks
| └─ foundry.toml
| └─ src/
| └─ test/
| └─ script/
└─ ops/
| └─ containerfiles/ # Podman build files per role
| └─ compose/ # podman-compose stacks
| └─ vmm/ # OpenBSD vmm vm definitions
| └─ gensyn/ # Gensyn distributed-training configs
└─ tests/ # pytest, persona regression
└─ docs/ # .py↔.md colocated where reasonable
└─ scripts/ # dev helpers

```

Hardware feasibility — VRAM math worked out per target

For **single H100 80 GB**, GLM-5.1 is impossible at any quantization (754 GB FP8 weights alone). Realistic targets are Qwen3-32B in BF16 (~64 GB weights + ~6 GB KV at 8 K + ~4 GB activations + grad + optimizer for inference, but training requires ZeRO offload), Qwen3-7B in BF16 with full LoRA (14 GB weights + LoRA adapters ~200 MB + Adam state ~28 GB for FP32 master + 14 GB grads — tight, use QLoRA), Phi-4-mini in BF16 with full SFT (~7.6 GB weights + 15.2 GB grads + 30.4 GB Adam states ≈ 53 GB, fits with margin), Gemma 4 31B with QLoRA. Active ~10B-class MoEs like Qwen3.5-35B-A3B fit in inference at INT4 (~17.5 GB weights + 8 GB KV + activations ≈ 30 GB), and QLoRA fine-tuning is feasible at 16K context.

For **8× H100 80 GB cluster** (640 GB aggregate), GLM-5.1-FP8 is borderline — 754 GB weights does not fit even with ZeRO-3 splitting unless KV is compressed via MLA and you accept 16-bit gradient checkpointing with CPU offload (KTransformers-style); the realistic posture is "inference yes via FP8 MLA-compressed, training no." Qwen3-235B-A22B in BF16 fits trivially for inference (~470 GB weights) and is trainable via FSDP+ZeRO-3 with QLoRA (active 22 B → adapter math is reasonable). Qwen3.5-122B-A10B in BF16 (~244 GB) is comfortable for both inference and full SFT. Mistral Large 3 (675 B / 41 B-A) at FP8 is comparable to GLM-5.1 — borderline. The 8×H100 sweet spot for full SFT is in the 32B-122B-active range.

For **single A100 80 GB**, treat as a slightly slower H100 with the same memory ceiling. GLM-5.1 still impossible. Qwen3-32B QLoRA feasible. Same VRAM math as H100.

For **8× A100 80 GB**, identical capacity to 8× H100 (640 GB) but lower throughput; same model ceilings.

For **RTX 4090 24 GB**, GLM-5.1 impossible. Qwen3-7B QLoRA feasible (4-bit weights ~3.5 GB + LoRA ~200 MB + KV at 4 K ~1 GB + grads + Adam ~18 GB). Phi-4-mini full SFT feasible (BF16, ~16 GB total). Qwen3-1.7B and Qwen3-0.6B full SFT comfortable. Edge fine-tunes only.

For **Apple Silicon M3 Ultra 192 GB unified memory** via MLX, GLM-5.1 impossible until `mlx-lm` issue #879 lands and INT4 quants become available (then INT4 + MLA-compressed KV ~383 GB at 128K context — *still* doesn't fit). Qwen3-32B BF16 fits with margin (~64 GB + KV). Qwen3.5-122B-A10B in INT4 (~30 GB weights) fits comfortably for inference; QLoRA fine-tuning works via MLX-LM's PEFT support. Mistral Large 3 at INT4 borderline (~169 GB + KV). M2 Max 96 GB is more constrained: Qwen3-32B INT4 (~16 GB) + LoRA fine-tune is the realistic ceiling.

For **Gensyn distributed training** (the \$5K hackathon track), the framework's posture is to use Gensyn's RL Swarm SDK over the WAN training fabric for the agentic-trajectory RL phase only — pretraining and SFT happen on owned/leased H100/A100 clusters; the Gensyn integration enters at the GRPO stage where many small rollout workers (Qwen3-7B / Phi-4-mini scale) generate trajectories that the central trainer aggregates. This both fits the Gensyn programming model and exercises the BANKON x402 micropayment rail when each rollout worker is paid per accepted trajectory.

Model size selection logic for mindX cognitive API

The mindX cognitive API at `mindx.pythai.net` should serve a tiered family, not a single flagship. The recommended composition: **edge tier** is `aGLM-BANKON-edge` derived from Phi-4-mini (3.8 B, MIT, BFCL 70.3) for sub-second-latency tool-call routing and on-device deployments; **mid tier** is `aGLM-BANKON-mid` derived from Qwen3-7B or Qwen3.5-Flash for the bulk of agentic traffic where per-token cost matters; **flagship tier** is `aGLM-BANKON-flag` derived from Qwen3.5-122B-A10B for hard agentic flows where 22 B-class active capacity isn't enough but 41 B-class is overkill; **specialist tier** is `aGLM-BANKON-spec` derived from GLM-5.1-FP8 for the long-horizon SWE / 8-hour-autonomous-session use case where GLM-5.1's SWE-Bench Pro 58.4 dominance and 200K-context DSA are decisive. Routing between tiers happens at the operator layer based on task classification (tool-call routing → edge; chat with tools → mid; complex agentic flow → flagship; long-horizon SWE → specialist).

Data pipeline

The dataset construction pipeline is a Pydantic-typed DAG: `curate` (pull from source — HF Datasets, Common Crawl-derived, codephreak conversation history, mindX session logs)

produces `raw/`; `dedupe` runs MinHash near-duplicate detection plus exact-match removal producing `deduped/`; `filter` applies language detection (`fasttext`), quality classifiers (Cosmopedia-style), and toxicity filtering producing `filtered/`; `tokenize` runs the target tokenizer with vocab-hash recorded into the manifest producing `tokenized/`; `pack` does sequence packing to the target context length producing `packed/`; `synth` is the optional synthetic-data generation step that uses GLM-5.1 (specialist tier) or Qwen3.5-122B-A10B (flagship tier) to generate domain-specific tool-use trajectories, persona-conditioned dialogues, and edge-case examples (the ml-intern healthcare demo's "generate 1,100 synthetic edge cases and upsample 50×" pattern is the template); `verify` produces a manifest containing every artifact's BLAKE3 hash, the Lighthouse-Filecoin CID, the source URLs, and the generation parameters. The whole pipeline is a single `mindxtrain.data.run(config)` call that produces a `DatasetManifest` Pydantic record consumed by `train`.

Evaluation harness

The eval harness has three sub-layers: standard benchmarks via `lighteval` and `inspect-ai` adapters (MMLU-Pro, GPQA-Diamond, AIME, MATH, HumanEval, MBPP, IFEval); agentic benchmarks via custom adapters (BFCL v3/v4 — full Berkeley harness wired in, τ^2 -Bench, τ^3 -Bench, AgentBench, GAIA, MCP-Atlas where dataset is public, SWE-Bench Verified — Pro is gated behind Scale AI submission); and the **persona-and-agenda regression suite**, which is unique to mindXtrain and irreplaceable. The persona suite tests "does the model still call itself codephreak", "does the model preserve the Professor Codephreak ML/CS/programming domain", "does the model stay agenda-conditioned when given a multi-step build task", "does the model emit the correct chat-template tokens for the target backend", and "does the model degrade gracefully when the agenda is unfulfillable." Regression detection is automatic: every checkpoint runs the full suite, scores are diffed against the baseline (the most recent green checkpoint), and any score regression beyond a configurable tolerance halts the deployment pipeline. Model card auto-generation reads the `TrainingRun` manifest plus eval JSONs and emits a Hugging Face-compatible `README.md` with full provenance.

Training pipeline configurations — copy-pasteable

A canonical Qwen3.5-122B-A10B SFT-LoRA configuration in TRL/PEFT, expressible directly in the mindXtrain JSON config:

```
json
```

```
{
  "run_id": "aGLM-BANKON-flag-sft-001",
  "base_model": {
    "id": "Qwen/Qwen3.5-122B-A10B",
    "revision": "main",
    "license": "apache-2.0",
    "vocab_hash": "blake3:..."
  },
  "tokenizer": {"chat_template": "qwen3"},
  "dataset": {
    "manifest_cid": "lighthouse://bafy...codephreak-sft-v3",
    "format": "jsonl-chatml",
    "max_seq_len": 16384,
    "packing": true
  },
  "trainer": {
    "type": "sft",
    "framework": "trl",
    "lora": {"r": 64, "alpha": 128, "dropout": 0.05,
      "target_modules": ["q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"]},
    "qlora": {"bits": 4, "compute_dtype": "bfloat16",
      "double_quant": true, "quant_type": "nf4"}
  },
  "optim": {
    "optimizer": "paged_adamw_32bit",
    "lr": 2e-5, "lr_scheduler": "cosine", "warmup_ratio": 0.03,
    "weight_decay": 0.0, "max_grad_norm": 1.0,
    "epochs": 3, "global_batch": 64, "micro_batch": 1,
    "grad_accum": 8, "grad_checkpointing": true
  },
  "distributed": {
    "strategy": "fsdp", "shard": "full", "mixed_precision": "bf16",
    "cpu_offload": false
  },
  "callbacks": {
    "eval_steps": 200, "save_steps": 500,
    "eval_suites": ["bfcl_v4", "persona_regression", "agenda_regression"],
    "stop_on_regression": true
  },
  "storage": {"provider": "lighthouse",
    "checkpoint_dir": "lighthouse://aGLM-BANKON-flag/sft-001/"},
  "provenance": {"erc8004_attest": true,
```

```
"x402_settlement": false,  
"algorand_asa": 203977300}  
}
```

For DPO, swap `trainer.type` to `dpo`, point the dataset at a preference manifest with `chosen/rejected` pairs, drop the LR to `5e-7`, set `beta: 0.1`, and target the same LoRA modules — TRL's `DPOTrainer` consumes this directly.

For GRPO over agentic trajectories, `trainer.type: "grpo"` with `reward_funcs: ["bfcl_pass", "tau_bench_score", "persona_consistency"]`, `num_generations: 8`, `temperature: 0.9`, `max_prompt_length: 8192`, `max_completion_length: 8192`. The reward functions are pluggable Python callables registered in `mindxtrain.train.grpo.reward_registry`.

Part 5 — automindX v2 integration

automindX v2 is the consumer of mindXtrain-produced `aGLM-BANKON-*` checkpoints. The upgrade path from `pythaiml/automindx/aglm.py` to v2 is mechanical given the gap analysis. The single class `LlamaModel(model_name, models_folder)` becomes the package `automindx.models.{Backend}` with backends `HfTransformersBackend`, `LlamaCppBackend`, `OllamaBackend`, `VllmBackend`, `OpenAiCompatBackend`, `ZaiBackend`, `AnthropicBackend`, `GroqBackend` — registered in `automindx.models.registry.ModelRegistry`, dispatched by string key, picking the right `automindx.chat.ChatTemplate` for the backend and falling back to `tokenizer.apply_chat_template`. The synchronous `generate_contextual_output` becomes `generate_contextual_output(context: ConversationContext) -> Generation`, with an `agenerate_contextual_output` async sibling — sync default surface is preserved so existing call sites work unchanged. The JSON-on-disk memory becomes a `MemoryStore` interface with backends `JsonFsBackend` (default, never break), `PgVectorScaleBackend`, `LighthouseFilecoinBackend`. The 4096-character ceiling (`chunk4096.py`) is replaced by tokenizer-aware truncation plus sliding-window summarization for long contexts; default ctx 8K-128K depending on backend. The hard-coded persona becomes `prompts/codephreak.yaml` with the persona, the agenda field as a first-class slot, and the chat-template-token mapping declared.

The model registry, versioning, and hot-swap mechanics: `automindx.deploy.Registry` is a content-addressed registry where every `aGLM-BANKON-*` checkpoint is identified by the BLAKE3 hash of its safetensors plus its `TrainingRun` manifest CID. The registry tracks `current`, `canary`, and `rollback` pointers per tier (edge, mid, flagship, specialist). Hot-swap is atomic: `automindx.deploy.HotSwap.promote(tier, run_id)` flips the `current` pointer after running the persona-regression and agenda-regression suites against the live API harness; on failure, it auto-rolls back. A/B testing is a traffic split at the `automindx.api.Router` layer: 95% to `current`, 5% to `canary`, with both branches logged as Claude-Code-JSONL trajectories to Lighthouse for

offline statistical comparison via `automindx.eval.ab_compare(run_a, run_b, metric)`. The `mindx.pythai.net` public API exposes an OpenAI-compatible `/v1/chat/completions` plus a mindX-native `/v1/agentlc` that takes an agenda field directly and returns a session ID plus streaming events. Every accepted request is provenance-linked: the model checkpoint hash, the run-id, the registry's ERC-8004 attestation, and (for paid tiers) the x402 micropayment receipt are all included in response headers.

Part 6 — comparative due diligence: the verdict

The brief asked for a comparison of GLM-5.1 against Qwen3 (235B / 72B / 32B / 14B / 7B / 4B / 1.7B / 0.6B), Llama 3.3 70B, DeepSeek V3.1, Mistral Large 2, Gemma 3, and Phi-4. The May-2026 reality has moved beyond several of those: Qwen3.5 (Flash, 27B-dense, 35B-A3B, 122B-A10B) ships and outperforms Qwen3-235B on multiple axes; Llama 4 (Scout/Maverick, April 2025) ships under the *Llama 4 Community License* with the 700M MAU clause and EU AUP; DeepSeek V3.2 / V3.2-Speciale (December 2025) ships under MIT; Mistral Large 3 + Ministral 3 (December 2025) ships under Apache 2.0; Gemma 4 (April 2026) ships under Apache 2.0 — the single biggest license unlock of 2026; Phi-4 / Phi-4-mini ships under MIT.

Apache-2.0 redistribution requires the upstream to be permissive: Apache 2.0, MIT, or BSD-class. That filter cleanly admits **GLM-5.1, Qwen3, Qwen3.5, DeepSeek V3.2, Mistral Large 3 + Ministral 3 + Mistral Small 4, Gemma 4, Phi-4, Phi-4-mini, and Yi-1.5**, and cleanly rejects **Llama 3.3, Llama 4, Cohere Command A** (CC-BY-NC, non-commercial), **Falcon 3** (TII Falcon 2.0 with AUP — most counsel will flag this as restrictive), and **Yi-Lightning** (proprietary API-only). That kills five of the ten brief candidates before benchmarks are scored.

The verdict, on the dimensions of license-compatibility, agentic capability, tool-use quality, fine-tunability, ecosystem support, performance-per-parameter, multilingual balance, and size-ladder coverage, is that **mindXtrain should target Qwen3.5 as the primary base and treat GLM-5.1 as a premium specialist track**. Six reasons.

First, license strength is symmetric. MIT and Apache 2.0 are functionally equivalent for downstream Apache 2.0 redistribution; both pass cleanly. There is no license-based reason to prefer GLM-5.1.

Second, GLM-5.1's wins are real but narrow. SWE-Bench Pro 58.4 is genuinely SOTA. Terminal-Bench 2.0, MCP-Atlas, BrowseComp, CyberGym, AIME 2026 95.3, GPQA-Diamond 86.2 are all top-tier. But for tool-use plus function-calling plus web automation — the brief's stated agentic spec — GLM-5.1 is overkill. The 8-hour-autonomous-session capability is for software-engineering agents specifically.

Third, fine-tunability gap. GLM-5.1 was 27 days old at research time; Axolotl recipes are still maturing, LLaMA-Factory support is partial, Unsloth has GGUF but full LoRA/QLoRA flows are not battle-tested. Qwen3 and Qwen3.5 have day-one PEFT, Axolotl, LLaMA-Factory, Unsloth, and TRL support with hundreds of teams' worth of exercised DPO and GRPO recipes.

Fourth, per-parameter economics. GLM-5.1 at 754B/40B-A requires 4–8×H100 to *serve* and ZeRO-3 territory to fine-tune. Qwen3.5-122B-A10B has roughly comparable intelligence at one-quarter the active footprint and runs on 2×H100 comfortably; Qwen3.5-35B-A3B with 3B active reportedly exceeds Qwen3-235B-A22B. The agentic fine-tune sweet spot for mindXtrain is the 30B-122B-active class.

Fifth, size ladder. mindXtrain needs a *family*, not a single model — flagship for hard flows, mid-tier for production traffic, edge for cost-or-latency-sensitive deployments. Only Qwen has a contiguous Apache-2.0 family from 0.6B → 235B → 235B-Instruct-2507 → 235B-Thinking-2507 → 3.5 medium series. GLM-5.1 is one model. Family completeness alone tips this toward Qwen.

Sixth, BFCL leadership lives in the Qwen lineage. Qwen3-32B hits BFCL v3 75.7%; Qwen3.5-122B-A10B reaches BFCL v4 72.2%; the GLM-4.5 ancestor leads at 76.7%, but GLM-5.1 has not officially submitted to BFCL. For a function-calling-first agentic framework, Qwen has more *demonstrated* surface.

The recommendation, restated in concrete pin-the-version-and-go form: target Qwen3.5-Flash for the edge tier or Phi-4-mini if BFCL-at-tiny-size matters more than open-Apache-only purity (Phi-4-mini is MIT, equivalent for redistribution); target Qwen3-7B for mid-tier; target Qwen3.5-122B-A10B for flagship; maintain GLM-5.1-FP8 as the specialist track for long-horizon SWE flows where 8-hour autonomous sessions and SWE-Bench Pro 58.4 are decisive; keep DeepSeek V3.2 as the reasoning-with-tool-use track backup if Qwen3.5-Thinking-class falls short on internal evals; keep Mistral Large 3 + Ministral 3 as the EU-jurisdictional secondary track; keep Gemma 4 as a watch-list option once vLLM kernel optimization and Axolotl recipes mature (the early-release speed and stability issues should clear within 60–90 days); do not base anything on Llama 3.3, Llama 4, Cohere Command A, Falcon 3, or Yi-Lightning regardless of benchmark performance.

Conclusion: provenance chain and execution order

The end-to-end provenance chain is: **upstream open-weight base** (Qwen3.5-122B-A10B Apache 2.0 for primary, GLM-5.1-FP8 MIT for specialist) → **mindXtrain training pipeline** (data DAG with Lighthouse-anchored manifests, SFT + LoRA + DPO + GRPO + tool-use trajectory training, full eval harness with persona regression) → **aGLM-BANKON-*** **checkpoint** (Apache 2.0, NOTICE preserves upstream MIT / Apache, BLAKE3-hashed and Lighthouse-Filecoin-CIDed) → **ERC-8004 attestation** (Identity Registry `0x8004A169...` and Reputation Registry `0x8004BAa1...` on EVM via Foundry-tested contract calls, BANKON ASA `203977300` on Algorand for settlement) → **automindX v2 model registry** (content-addressed, hot-swap with canary and rollback, persona-and-agenda regression gates) → **mindx.pythai.net cognitive API** (OpenAI-compatible `/v1/chat/completions` plus mindX-native `/v1/agentic`, x402 micropayment for paid tiers, full provenance in response headers).

Execution order, the boring critical-path version: first, lock the license posture — pull `zai-org/GLM-5.1-FP8` to a self-hosted store, write the Apache-2.0 + MIT-NOTICE compliance bundle for the BANKON derivative, draft the `mindx.pythai.net` Terms of Service that does *not* inherit Z.ai's AUP. Second, stand up the Qwen3.5-122B-A10B fine-tuning rail end-to-end on whatever 8×H100 / 8×A100 capacity is available, validate it with a deliberately-trivial Codephreak-persona SFT to exercise every callback, every storage backend, every regression gate. Third, port `pythaiml/automindx/aglm.py` to `automindx.models.HfTransformersBackend` with the `ChatTemplate` abstraction, preserving the persona-as-agenda discipline and the four-axis decomposition, in a v2 branch that the v1 callers can opt into without breaking. Fourth, reimplement the five ml-intern operator patterns into `mindxtrain.operator.*` once HF posts the LICENSE on `huggingface/ml-intern` (issue #41), or right now if the patterns are clean-room reimplemented from the public API description without copying source. Fifth, wire the storage and provenance layers — Lighthouse-Filecoin first (CIDs anchored in `TrainingRun` manifests), ERC-8004 attestation second (Foundry-tested contract calls from `mindxtrain.provenance.erc8004`), x402 settlement last. Sixth, take the resulting `aGLM-BANKON-flag-001` to the Gensyn distributed-RL track for the GRPO trajectory phase, exercising the WAN training fabric and the x402 per-trajectory micropayment rail simultaneously.

The framing that matters for everything downstream: GLM-5.1 is a remarkable model, the strongest open-weights agentic-engineering base in May 2026, and a serious specialist track for mindXtrain. But it is not, on the rigorous evidence, the right *primary* base for a multi-tier Apache-2.0-redistributable agentic family. The right primary is Qwen3.5. Building the framework with that priority pair locked in — Qwen3.5 primary, GLM-5.1 specialist — is what makes mindXtrain a production-grade training framework rather than a single-model wrapper, and what gives `aGLM-BANKON` the family completeness it needs to serve every cell of the mindX cognitive API at `mindx.pythai.net`.